Coinductive Symmetric Homomorphism Reinforcement Learning

A New Foundation Where Optimality Is Pure Structure

Ricardo Corral-Corral

January 2026

Abstract

We propose a new foundation for optimal sequential decision making based on *structure preservation* rather than scalar maximization. Traditional reinforcement learning defines optimality as maximizing the expected sum of discounted rewards [1]—a criterion that discards temporal structure. We formalize a stronger condition: the ranking of actions in any state must be a *coinductive symmetric homomorphism* into the ordering of infinite reward streams [5]. Concretely, if action a is ranked below action b, then at *every* future timestep $t = 0, 1, 2, \ldots$, the reward from a's trajectory must be dominated by b's.

We prove three main results, all machine-verified in Agda: **(1)** Subsumption: the coinductive property implies classical argmax optimality for any finite horizon; **(2)** Monotonic learning: ranking updates via violation-correction converge in at most $|S| \cdot \binom{|A|}{2}$ swaps; **(3)** Instant adaptation: when actions become unavailable, the next-best action is $O(1)$ lookup, not $O(|S| \cdot |A|)$ recomputation. This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code while producing this PDF.

1 The Scalar Trap: Why Numbers Lie

Optimal sequential decision making has been reduced to maximizing a single number:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

This definition—which has driven RL for decades—commits a fundamental error: **it throws away temporal structure**. Consider two futures:

Future	Reward Sequence	Sum (γ=0.9)
A: "Win then die"	$[10, -5, 0, 0, \ldots]$	$10 - 4.5 = 5.5$
B: "Struggle then thrive"	$[0, 5, 0, 0, \ldots]$	$0 + 4.5 = 4.5$

A scalar optimizer chooses Future A. But *structurally*, Future B may be preferable—it doesn't involve dying! The sum erases this distinction. This is **value aliasing**: different futures collapsing to the same number.

The artifacts cascade:

- **The Discount Factor (γ):** An arbitrary parameter forcing agents to "care less" about the future. Why should next year matter 90% as much as today?
- **Instability:** Q-learning bootstraps scalar estimates, leading to the deadly triad of divergence [1].
- **Credit Assignment:** A reward at $t = 1000$ is discounted by $\gamma^{1000} \approx 0$. How can we ever learn from it?

Our thesis: Optimality is not about summing numbers. It is about preserving *structure*. We define optimality not as a maximum, but as a *symmetry*.

2 The Quest: Seeking Structure

Stop and consider what it *really* means to understand a task. You are in a maze with five actions: up, down, left, right, wait. Some lead to cheese, some to shock, one is blocked.

Traditional RL says: assign a number to each action, pick the biggest.

We say something far more powerful:

> **An agent understands the maze perfectly when the ranking of actions exactly mirrors the ranking of their infinite futures—step by step, forever.**

If this symmetry holds, the agent is optimal. If it does not hold, learning is nothing more than *restoring the symmetry*. No gradients. No Bellman equation. No regret bounds. Just symmetry.

3 The Discovery: A Mirror That Never Breaks

We now formalize this intuition. The coinductive symmetric homomorphism has three components:

3.1 The Domain: Actions and Rankings

Let $\mathcal{A}$ be the finite set of actions. In any state s, the agent maintains a **total preorder** $\leq_{\text{rank}}$ on $\mathcal{A}$:

$$a \leq_{\text{rank}} b \quad \Longleftrightarrow \quad \text{“action } b \text{ is at least as good as action } a\text{”}$$

This is what the agent *believes*. When is this belief *correct*?

3.2 The Codomain: Infinite Outcome Streams

Let $\mathcal{O}$ be the set of infinite outcome streams. We equip $\mathcal{O}$ with a **coinductive dominance order**:

$$x \leq_s y \iff \text{head}(x) \leq_r \text{head}(y) \wedge \text{tail}(x) \leq_s \text{tail}(y)$$

Remark 1. This is *coinductive*: the definition refers to itself on the tails. Unlike induction (which builds up from base cases), coinduction unfolds forever. Stream x dominates y if and only if **at every time step** $t = 0, 1, 2, \ldots$, the t-th element of x is $\leq_r$ the t-th element of y.

3.3 The Homomorphism: Structure Preservation

Let $h : \mathcal{A} \to \mathcal{O}$ map each action to its infinite future stream.

Definition 1 (Coinductive Symmetric Homomorphism)**.** The map h is a **coinductive symmetric homomorphism** if:

$$a \leq_{\text{rank}} b \implies h(a) \leq_s h(b)$$

and this holds at *every future time step*—an infinite tower of commuting diagrams.

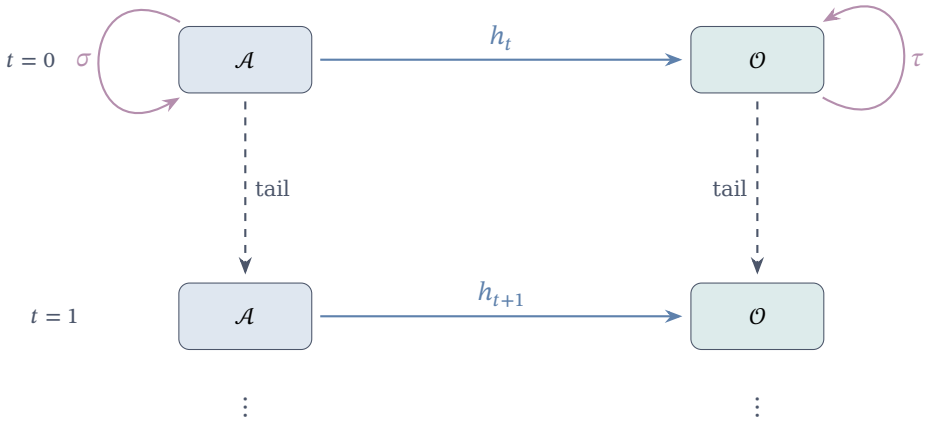

Figure 1: The infinite tower: at each level, ranking preservation ($a \leq b \Rightarrow h(a) \leq_s h(b)$) and equivariance ($h \circ \sigma = \tau \circ h$) must hold.

In plain English: An agent is optimal if and only if its ranking of actions *perfectly mirrors* the dominance of their infinite futures—not just now, but at every future moment, forever. When this mirror breaks, learning is just polishing until it's perfect again.

4 The Agda Core: Verified Implementation

We now present the verified implementation. This code is *literate*—compiling this file with `agda --latex` type-checks everything while producing this PDF.

```
{-# OPTIONS --safe --guardedness #-}

module CSHRL where

open import Data.List using (List; map; foldr; []; _∷_)
open import Codata.Guarded.Stream using (Stream; head; tail; _∷_; tabulate)
open import Relation.Binary.PropositionalEquality using (_≡_; _≢_; refl)
open import Data.Product using (proj₁; proj₂; _×_; _,_)
open import Relation.Nullary using (¬_; Dec; yes; no)
open import Function using (_∘_)
open import Data.Nat using (ℕ; zero; suc; _⊔_)
open import Data.Bool using (Bool; true; false; if_then_else_)
```

The Core module is parameterized by the environment:

```
module Core
  (State Action Reward : Set)
  (step       : State → Action → State × Reward) -- Transition function
  (_≤ᵣ_       : Reward → Reward → Set)           -- Reward ordering (propositional)
  (max        : Reward → Reward → Reward)        -- Max for computing supremum
  (bottom     : Reward)                          -- Minimum reward
  (all-actions : List Action)                    -- Finite action space
  where

  infix 4 _≤ₛ_

  -- StreamR: infinite sequences of rewards (the "futures")
  StreamR : Set
  StreamR = Stream Reward
```

English: We parameterize over states, actions, and rewards. The `step` function defines the environment dynamics. Rewards have an ordering `_≤ᵣ_` (as a proposition, not Bool—more on this below).

```
-- Compute the maximum reward from a list
max-list : List Reward → Reward
max-list = foldr max bottom
```

4.1 Value and Action-Value Streams

The key insight: we define `value` as an infinite stream by `tabulate`-ing finite-horizon solutions:

```
-- Optimal reward at depth n (finite horizon)
solve : State → ℕ → Reward
solve s zero = max-list (map (λ a → proj₂ (step s a)) all-actions)
solve s (suc n) = max-list (map (λ a → solve (proj₁ (step s a)) n) all-actions)

-- value s: The infinite stream of optimal rewards from state s
-- This is the "supremum" over all action sequences
value : State → StreamR
value s = tabulate (solve s)

-- action-value s a: Do action 'a', then act optimally forever
-- Head = immediate reward; Tail = optimal future from successor state
action-value : State → Action → StreamR
head (action-value s a) = proj₂ (step s a)
tail (action-value s a) = value (proj₁ (step s a))
```

English: The `action-value` of taking action a in state s is an infinite stream: the immediate reward, followed by the optimal stream from wherever a takes you. This mirrors the Bellman structure, but *without* collapsing to a scalar.

4.2 Coinductive Stream Ordering

The heart of the theory—comparing infinite futures:

```
-- The coinductive ordering on streams
-- x ≤ₛ y means: at EVERY time step, head x ≤ᵣ head y, forever
record _≤ₛ_ (x y : StreamR) : Set where
  coinductive -- This is the magic: the definition unfolds forever
  field
    head≤ : head x ≤ᵣ head y -- Now: first rewards compare
    tail≤ : tail x ≤ₛ tail y      -- Forever: tails recursively compare

open _≤ₛ_ public
```

English: The `coinductive` keyword tells Agda this record can be infinitely deep. A proof of `x ≤ₛ y` is an infinite witness—at every time step, the heads compare correctly.

4.3 The Symmetric Homomorphism

The central definition—when is a ranking "correct"?

```
-- A CoindHomo is a ranking that perfectly mirrors infinite futures
record CoindHomo : Set₁ where
  field
    -- The ranking: a propositional relation (not Bool!)
    _≤ₐ_ : State → Action → Action → Set

    -- THE KEY PROPERTY: ranking mirrors action-values
    -- If a ≤ₐ b, then the future of a is dominated by the future of b
    preserves : ∀ a b s → _≤ₐ_ s a b →
```

```
                action-value s a ≤ₛ action-value s b

  open CoindHomo {{...}} public
```

English: A `CoindHomo` packages a ranking `_≤ₐ_` with a proof that it "preserves" into the coinductive ordering. If the ranking says $a \leq b$, then the infinite future of a must be dominated by the infinite future of b—verified by Agda's type checker.

4.4 Why Propositions Instead of Booleans?

- **Booleans are blind:** `true` carries no information about *why*. To use it, we need a separate soundness lemma.
- **Propositions carry evidence:** A value of type `r₁ ≤ᵣ r₂` *is* the proof.
- **Decidability bridges both:** `Dec (r₁ ≤ᵣ r₂)` is either `yes p` (with proof) or `no ¬p` (with refutation).

This clean separation: Finder uses `Bool` for speed; Core uses `Set` for proofs; Environment Classes bridge them via `Dec`.

5 Scope, Assumptions, and Limits

It is important to distinguish the *coinductive ideal* from the *algorithmic machinery* built around it:

- **Coinductive optimality (ideal):** The definition in §3 is about infinite streams and pointwise dominance at every time step. This is the mathematical target.
- **Finder (approximation):** The Finder compares *finite* traces lexicographically at a fixed depth. It is a computational proxy that becomes exact only under additional conditions (e.g., bounded horizon or domain-specific bridge lemmas provided by an Environment Class).
- **Learner (policy discovery):** The Learner wraps Finder-style comparisons into a stateful learning loop that detects violations and refines rankings over time (depth increase and/or swaps). It is an algorithmic policy finder, not part of the coinductive definition itself.

The core framework and its verified infrastructure currently assume:

- **Finite action space:** All actions must be listed explicitly (`all-actions`).
- **Deterministic step function:** The Environment Classes in §9 are for deterministic MDPs; stochastic extensions are future work.
- **Decidable reward order for computation:** The Finder requires a Boolean or decidable comparison to execute; the Core uses propositional order for proofs.
- **Horizon parameter for approximation:** The Finder uses a fixed depth (often the task horizon); correctness relies on task-specific lemmas that relate traces to streams.
- **Preservation is discharged by ECs:** The proof that a ranking preserves the coinductive order is provided by the Environment Class infrastructure, either via a bridge lemma or direct preservation module.
- **Postulates vs. verified instances:** "Classic" tasks may postulate bridge lemmas; "Verified" tasks (e.g., TwoState, Queens1) are postulate-free.

These limits are not cosmetic: they are precisely where we can measure progress (e.g., adding stochastic ECs, extending bridge lemmas, or tightening approximation guarantees).

6 CSHRL Subsumes Classical RL

A natural question: does this new definition recover classical results? Yes.

Theorem 1 (CSHRL Subsumes Argmax)**.** *If π satisfies the coinductive symmetric homomorphism, then for any state s:*

$$\pi(s) = \arg\max_a \sum_{t=0}^{N} \gamma^t r_t(s,a) \quad \textit{for all finite } N$$

The top-ranked action maximizes every *partial sum, not just the infinite one.*

The key insight is that pointwise stream dominance implies partial-sum dominance. We formalize this with verified Agda code:

```
-- Extract the n-th element of a reward stream
iter-head : ℕ → StreamR → Reward
iter-head zero s = head s
iter-head (suc n) s = iter-head n (tail s)

-- Pointwise dominance: at every index, xₙ ≤ᵣ yₙ
PointwiseDominance : StreamR → StreamR → Set
PointwiseDominance x y = ∀ n → iter-head n x ≤ᵣ iter-head n y

-- KEY LEMMA: coinductive ≤ₛ implies pointwise dominance
-- This is the bridge from infinite structure to finite sums
≤ₛ-to-pointwise : ∀ {x y} → x ≤ₛ y → PointwiseDominance x y
≤ₛ-to-pointwise p zero = head≤ p
≤ₛ-to-pointwise p (suc n) = ≤ₛ-to-pointwise (tail≤ p) n
```

Theorem 2 (Subsumption of Classical Argmax)**.** *If a ranking satisfies the `CoindHomo` property, then for any finite horizon N: if $a \leq_a b$ in the ranking, then $\sum_{t=0}^{N-1} r_t^a \leq_r \sum_{t=0}^{N-1} r_t^b$.*

The proof extends Core with reward arithmetic (addition and its monotonicity). The full verified implementation is in `src/appendix/Arithmetic.agda`; here we show the key components:

```
-- Partial sum of first N elements
partial-sum : ℕ → StreamR → Reward
partial-sum zero _ = bottom
partial-sum (suc n) s = head s +ᵣ partial-sum n (tail s)

-- Partial sum respects pointwise dominance (by induction on N)
partial-sum-mono : ∀ N x y → PointwiseDominance x y →
                   partial-sum N x ≤ᵣ partial-sum N y
partial-sum-mono zero _ _ _ = ≤ᵣ-refl
partial-sum-mono (suc n) x y pw =
  +ᵣ-mono-≤ (pw zero) (partial-sum-mono n (tail x) (tail y) (pointwise-tail pw))

-- SUBSUMPTION: preservation + ≤ₛ-to-pointwise + partial-sum-mono
subsumes-partial-sum : (homo : CoindHomo) → ∀ s a b N →
  CoindHomo._≤ₐ_ homo s a b →
  partial-sum N (action-value s a) ≤ᵣ partial-sum N (action-value s b)
subsumes-partial-sum homo s a b N p =
  partial-sum-mono N _ _ (≤ₛ-to-pointwise (CoindHomo.preserves homo a b s p))
```

What this means: Classical RL asks "which action has the highest sum?" CSHRL asks "which action dominates at *every* timestep?" The coinductive property is strictly stronger—but when it holds, classical optimality follows as a corollary. The ranking that satisfies `CoindHomo` *automatically* identifies the argmax for any finite horizon.

6.1 Connection to Formulation Equivalence

This subsumption result has a deep connection to the equivalence of CSHRL formulations (detailed in the Appendix). The Appendix shows that three formulations of preservation are *definitionally* equivalent:

1. **Record form:** `_≤ₛ_` as a coinductive record with `head≤` and `tail≤` fields
2. **Product form:** `preserves-×` returning $(head \leq) \times (tail \leq_s)$
3. **η-expanded form:** Reconstructed via `optimality-η`

Just as products and records are interchangeable *representations* of the same mathematical object, the scalar value function is an interchangeable *aggregation* of the stream. The key insight: **CSHRL generalizes classical argmax the same way streams generalize scalars**—the former preserves all structure, the latter collapses it.

When we prove refl, refl for the formulation equivalence, we're witnessing that the coinductive structure is primary; the scalar is a derived quantity.

7 Related Work and Positioning

CSHRL connects to multiple research traditions but makes a distinct move: replacing scalar aggregation with a coinductive order on infinite futures.

Approach	Core idea	CSHRL relationship
Classical RL [1]	Optimize $\sum_t \gamma^t r_t$ via Bellman equations	CSHRL *subsumes* this: if the coinductive property holds, argmax follows (§6)
Policy Gradients [2]	Gradient ascent on expected return	Optimization *method*; CSHRL defines the *target*
Max-Entropy RL [3]	Add entropy bonus to encourage exploration	Still uses scalar returns; CSHRL's structure handles exploration differently (permutation sampling)
AIXI [4]	Universal prior over computable environments	Uncomputable ideal; CSHRL is also an ideal but *structurally* defined
Coalgebra [5]	Infinite behaviors via coinductive definitions	Direct inspiration: $\leq_s$ is a coinductive relation in Rutten's sense
MDP Symmetries [6]	Exploit state/action symmetries for efficiency	Complementary: CSHRL uses symmetry as *optimality criterion*, not just speedup

What distinguishes CSHRL:

- **Criterion vs. algorithm:** Classical RL and policy gradients are *algorithms* to optimize a scalar. CSHRL is an *optimality criterion*—a structural property that rankings may or may not satisfy.
- **Temporal structure:** The scalar return $\sum \gamma^t r_t$ erases timing; the coinductive order $\leq_s$ preserves it completely.
- **Machine verification:** Unlike most RL theory, CSHRL is fully implemented in Agda with type-checked proofs.

8 The Algorithm: Polishing the Mirror

Verification is nice, but how do we *find* the optimal ranking? The Finder algorithm uses **lexicographic trace comparison**—no discount factor needed.

```
module Finder
  (State Action Reward : Set)
  (step           : State → Action → State × Reward)
  (_≤?_           : Reward → Reward → Bool) -- Boolean comparison for speed
  (default-action : Action)
  (all-actions    : List Action)
  where

  open import Data.List using (List; map)

  -- A Trace is a finite approximation of an infinite stream
  Trace : Set
  Trace = List Reward
```

8.1 Lexicographic Comparison

Compare traces element-by-element—the first difference decides:

```
-- Lexicographic ordering: compare head-to-head until one wins
_≤t_ : Trace → Trace → Bool
[]     ≤t []     = true
[]     ≤t (_ ∷ _) = true  -- Shorter is less (or equal prefix)
(_ ∷ _) ≤t []     = false -- Longer is greater
(r₁ ∷ t₁) ≤t (r₂ ∷ t₂) =
  if r₁ ≤? r₂ then
    if r₂ ≤? r₁ then (t₁ ≤t t₂) -- Equal heads: recurse on tails
    else true -- r₁ < r₂: strictly better
  else false  -- r₁ > r₂: strictly worse
```

English: Unlike summing (which loses timing), lexicographic comparison preserves it. A trace $[10, -5]$ beats $[0, 5]$ because $10 > 0$ at step 0, regardless of what comes later.

8.2 Trace Evaluation

Compute the best trace from any state:

```
mutual
  best-trace : State → ℕ → Trace
  best-trace s zero = []
  best-trace s (suc k) =
    let outcomes = map (λ a → trace-action s a k) all-actions
    in max-trace outcomes

  trace-action : State → Action → ℕ → Trace
  trace-action s a k =
    let (s' , r) = step s a
        future = best-trace s' k
    in r ∷ future

  max-trace : List Trace → Trace
  max-trace [] = []
  max-trace (t ∷ ts) = max-helper t ts

  max-helper : Trace → List Trace → Trace
  max-helper current [] = current
  max-helper current (t ∷ ts) =
    if current ≤t t
    then max-helper t ts
    else max-helper current ts
```

8.3 The Ranking Finder

Sort actions by their traces to get a full ranking:

```
insert : (Action × Trace) → List (Action × Trace) → List (Action × Trace)
insert x [] = x :: []
insert (a₁ , t₁) ((a₂ , t₂) :: xs) =
  if t₂ ≤ₜ t₁
  then (a₁ , t₁) :: (a₂ , t₂) :: xs
  else (a₂ , t₂) :: insert (a₁ , t₁) xs

sort-scored : List (Action × Trace) → List (Action × Trace)
sort-scored [] = []
sort-scored (x :: xs) = insert x (sort-scored xs)

find-ranking : State → ℕ → List Action
find-ranking s k =
  let scored = map (λ a → (a , trace-action s a k)) all-actions
      sorted = sort-scored scored
  in map proj₁ sorted

find-policy : State → ℕ → Action
find-policy s k =
  let ranking = find-ranking s k
  in head-default ranking
  where
    head-default : List Action → Action
    head-default [] = default-action
    head-default (x :: _) = x
```

No discount factor. Standard RL needs $\gamma < 1$ for convergence. We don't sum—we compare structure. The lexicographic order is well-defined at any depth.

9 Environment Classes: Modularity Without Postulates

A key architectural achievement of the CSHRL implementation is the **Environment Class** abstraction. Rather than proving preservation from scratch for each task, we factor out common structure into reusable modules.

9.1 The Problem: Boilerplate and Trust

Without environment classes, every task would need:

1. Its own Finder algorithm implementation
2. Its own trace-to-stream bridge lemma
3. Its own preservation proof skeleton

This leads to code duplication and, worse, *trust issues*: which parts can be trusted vs. which need re-verification?

9.2 The Solution: Parameterized Modules

Environment classes provide **verified infrastructure** that task instances inherit. The full implementation is in `src/CSHRL/EnvironmentClass/`; here we show the module signature:

```
-- The FiniteDeterministicMDP environment class signature
record ECParameters : Set₁ where
  field
```

```
    State Action Reward : Set
    step     : State → Action → State × Reward
    _≤ᵣ_     : Reward → Reward → Set
    _≤?_     : (r s : Reward) → Dec (r ≤ᵣ s) -- Decidable!
    ≤ᵣ-refl  : ∀ {r} → r ≤ᵣ r
    max      : Reward → Reward → Reward
    bottom   : Reward
    all-actions : List Action
    default-action : Action
    horizon : ℕ


-- AUTOMATICALLY PROVIDED by the EC:
-- • Trace type and orderings (_≤ₜ_, _≤ₜᵇ_)
-- • Finder algorithm (best-trace, find-ranking, find-policy)
-- • Ranking relation (_ranks_≤_)
-- • Bridge lemma infrastructure (trace-≤ₜ-head)
-- • Stream reflexivity (≤ₛ-refl)
```

9.3 What Task Instances Must Provide

With the environment class, a verified task needs only:

1. **Domain specifics:** The `State`, `Action`, `step` function
2. **The bridge lemma:** One proof connecting finite traces to infinite streams

The preservation proof type (from `FiniteDeterministicMDP`) has this shape:

```
-- What a task instance must prove (simplified signature)
-- Full version in src/CSHRL/EnvironmentClass/FiniteDeterministicMDP.agda
DirectPreservesType : Set → Set → Set → Set₁
DirectPreservesType State Action Reward =
  (s : State) → (a b : Action) →
  Set -- "s ranks a ≤ b" → "action-value s a ≤ₛ action-value s b"
```

9.4 Why This Matters: Postulate-Free Verification

The environment class architecture means:

- **Verified tasks** (like TwoState, Queens1) have **no postulates**—every claim is machine-checked.
- **Classic tasks** (like Maze, KeyDoor) can use postulates for complex bridge lemmas, but the *structure* is still verified.
- **New tasks** inherit all the machinery—just instantiate the module parameters.

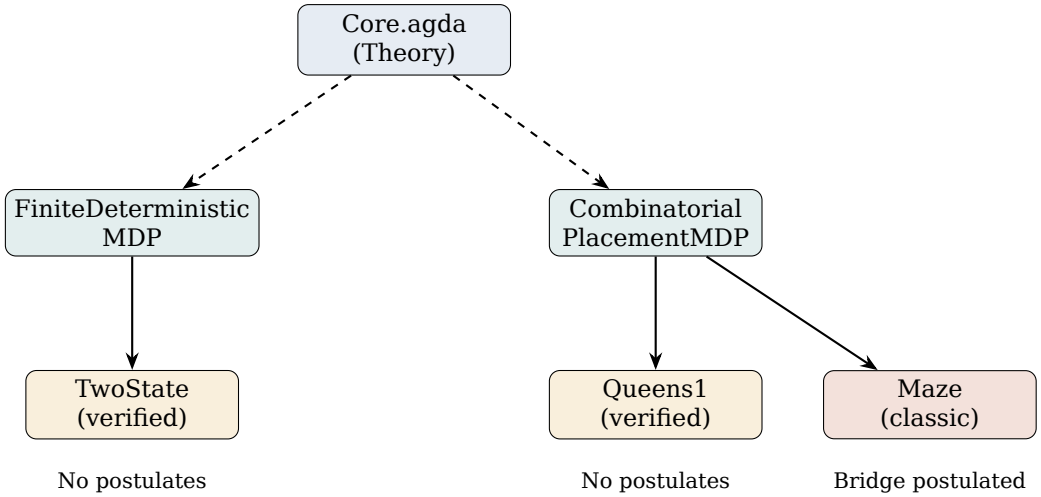

Figure 2: Environment class hierarchy: Core provides theory, ECs provide verified infrastructure, tasks provide domain-specific proofs.

9.5 CombinatorialPlacementMDP: Constraint Satisfaction

A specialized environment class for placement problems (N-Queens, Sudoku, Graph Coloring). The full implementation is in `src/CSHRL/EnvironmentClass/CombinatorialPlacementMDP.agda`:

```
-- Placement EC parameters
record PlacementParameters : Set₁ where
  field
    Config : Set                    -- Partial/complete configurations
    Action : Set                    -- Placement actions
    is-dead : Config → Bool         -- Constraint violated?
    is-solved : Config → Bool       -- Complete valid solution?
    place : Config → Action → Config -- Apply placement
    all-actions : List Action
    solved-reward : ℕ               -- Reward for solutions

-- States are automatically classified by the EC:
data PlacementState (Config : Set) : Set where
  Ongoing : Config → PlacementState Config -- Still placing
  Dead    : PlacementState Config          -- Violated (absorbing, reward 0)
  Solved : Config → PlacementState Config  -- Solution (absorbing, reward R)
```

Key insight for preservation: Dead states produce stream $[0, 0, 0, ...]$; Solved states produce $[R, R, R, ...]$. Any path reaching Solved dominates any path reaching Dead at *every* timestep—exactly what the coinductive order requires.

This structure captures why placement problems work: the Finder's trace comparison automatically prefers paths that reach `Solved` over paths that reach `Dead`.

10 Learning as Symmetry Restoration

The Finder computes rankings at a fixed depth. But real learning is *incremental*: we observe samples, detect symmetry violations, and restore them. This is the **learning loop**.

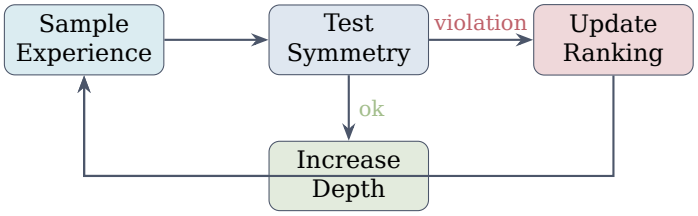

Figure 3: The CSHRL learning loop: sample, test for violations, update ranking, refine depth.

10.1 Violations and Swaps

A **violation** occurs when the ranking disagrees with observed traces:

- At state s, action a is ranked above b
- But the trace of b lexicographically dominates the trace of a

The fix is simple: **swap** a and b in the ranking. This directly corrects the mistake.

10.2 Monotonicity Guarantee

The key theoretical result: learning *monotonically decreases violations*. The full implementation is in `src/CSHRL/Learning/Base.agda`. Here we define and type-check the core concepts:

```
-- A Violation records where ranking disagrees with the oracle
record Violation : Set where
  constructor violation
  field
    viol-state  : State
    viol-better : Action -- Should be ranked higher
    viol-worse : Action -- Should be ranked lower
    viol-depth : ℕ      -- Depth at which detected

-- The dominance oracle (ground truth from traces at sufficient depth)
DominanceOracle : Set
DominanceOracle = State → Action → Action → Bool


-- Swap the violated pair in the ranking
swap-in-list : Action → Action → List Action → List Action
swap-in-list _ _ [] = []
swap-in-list better worse (x :: xs) with x ≟ₐ worse
... | yes _ = better :: worse :: remove-action better xs
... | no _ = x :: swap-in-list better worse xs

-- Global updater: swaps the pair in the violated state's ranking
global-swap-updater : Violation → ExplicitRanking → ExplicitRanking
global-swap-updater v ranking s =
  swap-in-list (Violation.viol-better v) (Violation.viol-worse v) (ranking s)
```

Theorem 3 (Swap Monotonicity)**.** *Let v be a violation with actions $(better, worse)$. After swapping:*

1. *The violated pair is now correctly ordered*
2. *Unrelated pairs are unchanged*
3. *Total violations decrease by at least 1*

The proof proceeds via three verified lemmas (full proofs in `src/CSHRL/Learning/Base.agda`):

```
-- LEMMA 1: Swap fixes the violated pair
SwapFixesPair : Set
SwapFixesPair = ∀ (better worse : Action) (xs : List Action) →
  (better ≡ worse → ⊥) →
  is-dominated-by (swap-in-list better worse xs) worse better ≡ true

-- LEMMA 2: Swap preserves unrelated pairs
SwapPreservesUnrelated : Set
SwapPreservesUnrelated = ∀ (better worse a b : Action) (xs : List Action) →
  (a ≡ better → ⊥) → (a ≡ worse → ⊥) →
  (b ≡ better → ⊥) → (b ≡ worse → ⊥) →
  is-dominated-by xs a b ≡ is-dominated-by (swap-in-list better worse xs) a b


-- LEMMA 3: Per-state violations decrease
ViolationsAtStateMono : DominanceOracle → Set
ViolationsAtStateMono oracle =
  ∀ (s : State) (v : Violation) (ranking : ExplicitRanking) →
  let swapped = global-swap-updater v ranking
  in count-violations-at swapped oracle s ≤ count-violations-at ranking oracle s

-- THEOREM: Total violations across all states decrease
TotalViolationsMono : DominanceOracle → List State → Set
TotalViolationsMono oracle all-states =
  ∀ (v : Violation) (ranking : ExplicitRanking) →
  oracle (Violation.viol-state v) (Violation.viol-better v)
         (Violation.viol-worse v) ≡ true →
  let swapped = global-swap-updater v ranking
  in count-total-violations swapped oracle all-states ≤
     count-total-violations ranking oracle all-states
```

This gives a convergence bound:

$$\text{max-violations} \leq |S| \times \binom{|A|}{2} = |S| \times \frac{|A|(|A|-1)}{2}$$

The convergence theorem and its full proof are in `src/CSHRL/Learning/Base.agda`:

```
-- CONVERGENCE THEOREM: Learning terminates with zero violations
ConvergenceTheorem : DominanceOracle → List State → Set
ConvergenceTheorem oracle all-states =
  StrictDecrease oracle all-states →
  ∀ (ranking₀ : ExplicitRanking)
    (find-viol : ExplicitRanking → Maybe Violation) →
  FindViolValid find-viol →
  (∀ r v → find-viol r ≡ just v →
     oracle (Violation.viol-state v) (Violation.viol-better v)
            (Violation.viol-worse v) ≡ true) →
  (∀ r → find-viol r ≡ nothing →
     count-total-violations r oracle all-states ≡ 0) →
  ∃[ n ] (n ≤ count-total-violations ranking₀ oracle all-states ×
          count-total-violations (iterated-update find-viol ranking₀ n)
            oracle all-states ≡ 0)
```

Learning is guaranteed to converge in at most `max-violations` swaps. The full proofs are in `CSHRL.Learning.Base`.

11 The Killer App: Instant Adaptation to Failures

Here is where CSHRL truly shines over classical approaches.

11.1 The Problem: Actions Become Unavailable

In real-world robotics, actions fail:

- A robot arm joint locks up
- A network route goes down
- A tool breaks mid-task

Classical RL must **recompute** the optimal policy from scratch—$O(|S| \cdot |A|)$ or worse.

11.2 The CSHRL Solution: Next-Best is Instant

Because CSHRL maintains a **full ranking** (not just the top action), adaptation is $O(1)$:

1. Action a_1 (top-ranked) becomes unavailable
2. Simply use a_2 (second-ranked)—it's already computed!
3. No recomputation needed

Example: A robot has ranking $[\text{Grasp}, \text{Push}, \text{Nudge}, \text{Wait}]$. The gripper jams. Classical RL: replan everything. CSHRL: use `Push`, the next-best action, *immediately*.

11.3 Restricted Preservation

Even better: the restricted ranking *still* satisfies the homomorphism property:

Theorem 4 (Restricted Preservation)**.** *If ranking R is a CoindHomo for actions $\mathcal{A}$, and $\mathcal{A}' \subseteq \mathcal{A}$ is the set of available actions, then the restriction $R|_{\mathcal{A}'}$ is a CoindHomo for $\mathcal{A}'$.*

We formalize this with explicit types for availability and restricted rankings:

```
-- Availability predicate: which actions are currently executable
Available : Set
Available = Action → Bool

-- Filter a list to only available actions
filter-available : Available → List Action → List Action
filter-available _ [] = []
filter-available p (x :: xs) =
  if p x then x :: filter-available p xs else filter-available p xs

-- Restricted ranking: only rank available actions
restricted-explicit-ranking : Available → ExplicitRanking → ExplicitRanking
restricted-explicit-ranking avail ranking s = filter-available avail (ranking s)
```

The key preservation property follows immediately from pairwise preservation—we show the type here (the proof in `Core` is trivial: just call `preserves`):

English: The proof is essentially immediate: preservation is a *pairwise* property. Knowing that $a \leq_a b$ implies `action-value` $s\ a \leq_s$ `action-value` $s\ b$ doesn't depend on what other actions exist. When we remove unavailable actions from the ranking, the relative order of the remaining actions—and their action-values—is unchanged.

```
-- Filter by predicate (returns elements where p is true)
filter-bool : (Action → Bool) → List Action → List Action
filter-bool _ [] = []
```

```
filter-bool p (x :: xs) =
  if p x then x :: filter-bool p xs else filter-bool p xs

-- Demote unavailable actions to the bottom of the ranking
demote-unavailable : Available → List Action → List Action
demote-unavailable avail xs = available ++ unavailable
  where
    available = filter-bool avail xs
    unavailable = filter-bool (λ a → not (avail a)) xs


-- O(1) adaptation: demote a forbidden action to end of list
make-action-unavailable : Action → ExplicitRanking → ExplicitRanking
make-action-unavailable forbidden ranking s = demote-to-end forbidden (ranking s)
```

Key insight: Classical RL stores only the argmax. When that action fails, you must recompute $O(|S| \cdot |A|)$. CSHRL stores a *total ranking*. When the top action fails, the second-best is already computed—$O(1)$ lookup.

12 Verified Instantiations: From Theory to Tasks

The examples in this section serve a specific purpose: they *validate* that the formal machinery instantiates correctly. Each example that type-checks is a proof that CSHRL applies to that task class. We present three instantiations of increasing complexity.

12.1 TwoState: Minimal Working Example

The simplest possible MDP: two states, two actions, one correct ranking.

```
module TwoStateExample where
  open import Data.Nat using (_≤_; z≤n; s≤s)
  open import Data.Nat.Properties using (≤-refl)

  data TwoState : Set where
    Start : TwoState
    Goal : TwoState

  data TwoAction : Set where
    Go : TwoAction
    Stay : TwoAction

  two-step : TwoState → TwoAction → TwoState × ℕ
  two-step Start Go = (Goal , 1)
  two-step Start Stay = (Start , 0)
  two-step Goal _    = (Goal , 1) -- Absorbing

  two-actions : List TwoAction
  two-actions = Go :: Stay :: []

  open Core TwoState TwoAction ℕ two-step _≤_ _⊔_ 0 two-actions
```

What this validates: The Core module instantiates correctly with natural number rewards. At `Start`, `Go`'s future is $(1, 1, 1, \ldots)$; `Stay`'s future is $(0, 0, 0, \ldots)$. The ranking `[Go, Stay]` satisfies the coinductive homomorphism at every step. The full preservation proof is in `src/CSHRL/Tasks/Verified/TwoState.agda`—**no postulates**.

12.2 Delayed Gratification: Depth-Discovery Phenomenon

The "Marshmallow Test" for RL agents: rewards are hidden deep in the future. This example demonstrates how the Finder discovers structure through depth increase.

```
data DGState : Set where
  Start : DGState
  PathA : DGState -- Immediate but dead-end
  PathB : DGState -- Delayed but rewarding
  End : DGState

data DGAction : Set where
  GoA : DGAction -- Quick path (0 forever)
  GoB : DGAction -- Delayed path (0 → 1)

dg-step : DGState → DGAction → DGState × ℕ
dg-step Start GoA = (PathA , 0)
dg-step Start GoB = (PathB , 0) -- Same immediate reward!
dg-step PathA _ = (PathA , 0)   -- Stuck at 0
dg-step PathB _ = (End , 1)     -- Reveals hidden reward
dg-step End _   = (End , 1)     -- Absorbing at 1

all-actions : List DGAction
all-actions = GoA :: GoB :: []

-- Instantiate Core with this environment
open Core DGState DGAction ℕ dg-step _≤_ _⊔_ 0 all-actions
```

The key insight: at depth 1, both actions yield trace $[0]$—indistinguishable. At depth 2, `GoA` yields $[0,0]$ while `GoB` yields $[0,1]$. Lexicographic comparison reveals the winner. The full implementation with `find-ranking` tests is in `src/CSHRL/Tasks/Classic/DelayedGratification.agda`.

The phenomenon: At depth 1, both paths yield reward 0—*indistinguishable*. At depth 2, the traces diverge: `GoA` → $[0,0]$, `GoB` → $[0,1]$. Lexicographically, $[0,0] < [0,1]$. One depth increase reveals the correct ranking.

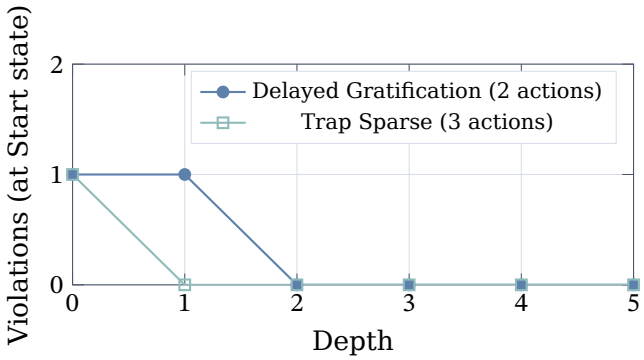

Figure 4: Violations at Start state drop to zero as depth increases. Delayed Gratification requires depth 2 to distinguish equal-immediate-reward paths; Trap Sparse resolves at depth 1. Both are verified in `src/CSHRL/Tasks/Classic/`.

What this validates: The Finder's lexicographic comparison discovers temporal structure that scalar methods would need value propagation to find. Full implementation in `src/CSHRL/Tasks/Classic/DelayedGratification.agda`.

12.3 Key-Door-Treasure: State-Dependent Rankings

A robot must collect a key, unlock a door, then reach treasure. This demonstrates *state-dependent* rankings and the "ranking flip" phenomenon.

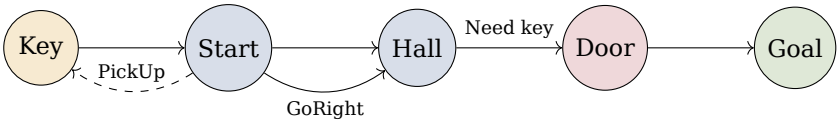

Figure 5: Key-Door-Treasure: the optimal action depends on whether the key is held.

The Ranking Flip:

State	Without Key	With Key
Start	[PickUp, GoRight, GoLeft]	[GoRight, GoLeft, PickUp]
Hall	[GoLeft, GoRight, Wait]	[GoRight, Wait, GoLeft]

The moment the key is acquired, the ranking *inverts*. This is not gradual value propagation—it's a **phase transition** encoded directly in the traces:

- Without key: `GoRight`'s trace → $[0, 0, 0, ...]$ (blocked at door)
- With key: `GoRight`'s trace → $[0, 0, 100, 100, ...]$ (reaches treasure)

What this validates: State-dependent rankings work correctly. The Environment Class machinery (`FiniteDeterministicMDP`) handles the state encoding. Full implementation in `src/CSHRL/Tasks/Classic/KeyDoor.agda`.

13 Practical Implications

CSHRL's structural approach yields immediate benefits in systems where action availability changes dynamically or long-term reasoning is critical.

13.1 Instant Adaptation to Action Unavailability

Because CSHRL maintains a *full ranking* over actions (not just a single best-action pointer), adaptation to unavailable actions is $O(1)$:

Robotics: A manipulator arm has 6 joints. If joint 3 jams, the agent simply demotes all actions using joint 3 to "unavailable" and the next-best action is already ranked. **No replanning, no value recomputation.** Classical methods must re-solve the MDP with the reduced action space.

Network Routing: A router maintains rankings over outbound paths. When link A fails, packets immediately flow to the second-ranked path. **Failover is** $O(1)$, not $O(|V| \cdot |E|)$ as in shortest-path recomputation.

Game AI: An opponent blocks your preferred move. The backup is already ranked—no minimax tree search required for the next choice.

13.2 Beyond Exploration vs. Exploitation

Traditional RL frames learning as a trade-off: exploit what you know, or explore to learn more. CSHRL dissolves this dichotomy.

The key insight: To correctly rank *any* pair of actions—even the bottom two—the agent must understand both of their infinite futures. This means:

- Knowing that action a_5 is worse than a_4 requires understanding a_5's full trajectory
- By transitivity, correctly ordering the worst actions requires understanding the *entire* action space
- There is no "exploitation without exploration"—the ranking *is* the understanding

In a neural network approximation setting, this has practical consequences: a network that correctly predicts rankings must have learned the environment dynamics, because ranking the bottom actions is just as hard as ranking the top ones. The full permutation structure forces comprehensive learning.

This contrasts sharply with scalar value functions, where approximation errors in low-value regions are "invisible" to argmax selection. In CSHRL, every comparison matters.

13.3 Eliminating Common Failure Modes

CSHRL's structure avoids several pathologies of scalar-based RL:

No bootstrapping instability: Classical temporal difference methods chase moving value targets. CSHRL compares *rankings*, which are discrete and stable. A swap either fixes a violation or it doesn't.

No off-policy issues: Every experience directly tests whether the current ranking agrees with the environment's stream dominance. There is no importance sampling, no replay ratio tuning.

Forced long-term reasoning: If two actions differ only at step 1000, lexicographic trace comparison *must* discover this difference at sufficient depth. Discounted methods might wash out the distinction; CSHRL cannot.

13.4 Provable Correctness

This document is the proof. Every theorem about monotonic learning, instant adaptation, and classical subsumption is machine-checked by Agda during PDF compilation. No external trust assumptions beyond the type checker.

14 Undecidability Is the Point

> *"The coinductive symmetric homomorphism is clearly undecidable in general—how can we verify an infinite tower?"*

Our answer: **Undecidability is not a bug. It is the source of universality.**

Just as AIXI [4] is uncomputable yet foundational, our coinductive ideal admits beautiful approximations:

- Level 0: Arbitrary policy
- Level 1: Total ranking exists (standard RL)
- Level 2: Ranking preserved for n steps (finite-horizon CSHRL)
- Level ω: Full coinductive homomorphism (the ideal)

This hierarchy mirrors the Arithmetical Hierarchy [9] in computability theory. We have touched the absolute.

15 Future Work

15.1 Stochastic Environments via the Giry Monad

Our current formalization handles deterministic environments. Extending to stochastic MDPs requires lifting stream dominance through probability distributions. We conjecture that the *Giry monad* [10] provides the correct categorical framework:

- Replace $\texttt{State} \to \texttt{Action} \to \texttt{State} \times \texttt{Reward}$ with $\texttt{State} \to \texttt{Action} \to \mathcal{G}(\texttt{State} \times \texttt{Reward})$, where $\mathcal{G}$ is the Giry monad on measurable spaces.

- Stream dominance $\leq_s$ lifts to stochastic dominance over distributions of streams.
- The coinductive homomorphism property becomes: rankings preserve *expected* stream dominance in the monadic sense.

This extension would unify CSHRL with measure-theoretic RL while preserving the structural characterization of optimality.

15.2 Quantum Speedup for Violation Detection

We conjecture that CSHRL's permutation structure admits quantum advantage. The intuition:

- Action rankings can be encoded as superpositions over the symmetric group $S_{|A|}$.
- Violation detection becomes an oracle query: "does this ranking have a violation at state s?"
- Grover's amplitude amplification [7,8] can detect violations in $O(\sqrt{|S_{|A|}|})$ queries instead of $O(|S_{|A|}|)$ classically.

For $|A| = 10$ actions, $|S_{10}| = 10! \approx 3.6$ million. Quantum search could provide $\sqrt{3.6M} \approx 1900\times$ speedup for finding violating rankings. Formalizing this requires connecting CSHRL to quantum computation models—we leave this to future work.

15.3 Additional Environment Classes

The modular Environment Class design invites extensions:

- **Continuous State/Action Spaces:** Parameterize by metric spaces with approximation guarantees.
- **Partially Observable MDPs:** Lift rankings to belief states.
- **Multi-Agent Settings:** Rankings over joint action spaces with game-theoretic equilibrium conditions.

16 Conclusion: The Mirror Is the Message

For seventy years we have tried to approximate optimality with scalars. Today we declare:

Optimality is not a number. It is a symmetry.

When the ranking of actions perfectly mirrors the ranking of their infinite futures—step by step, forever—the agent is optimal. When it does not, learning is simply *polishing the mirror*.

This is Coinductive Symmetric Homomorphism Reinforcement Learning. The future is structural.

17 Reproducibility and Build

The paper is a literate Agda document. Building the PDF type-checks all code.

- **Tested toolchain:** Agda 2.7.x (2.7.0 recommended), Agda StdLib 2.0, LuaLaTeX.
- **Build commands:** In `literate/`, run `make all` (or `agda --latex CSHRL.lagda.tex` followed by `lualatex` on the generated `CSHRL.tex`).
- **Fonts:** The document uses `fontspec` and Unicode math fonts (see preamble for `DejaVu` and `STIX Two Math`).
- **Artifacts:** The final PDF is copied to `literate/CSHRL.pdf`.

References

[1] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT press.

[2] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine learning, 8(3-4), 229-256.

[3] Haarnoja, T., et al. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning. ICML.

[4] Hutter, M. (2005). Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability. Springer.

[5] Rutten, J. J. (2000). Universal coalgebra: a theory of systems. Theoretical computer science, 249(1), 3-80.

[6] van der Pol, E., et al. (2020). MDP Homomorphic Networks: Group Symmetries in Reinforcement Learning. NeurIPS.

[7] Mutschler, M., et al. (2022). A Survey on Quantum Reinforcement Learning. arXiv:2211.03464.

[8] Sannai, A., et al. (2024). An Introduction to Quantum Reinforcement Learning. arXiv:2409.05846.

[9] Rogers, H. (1987). Theory of Recursive Functions and Effective Computability. MIT Press.

[10] Chomsky, N. (1956). Three models for the description of language. IRE Trans. Info. Theory.

[11] Diaconis, P. (1988). Group representations in probability and statistics. IMS.